# Cynbit Inquiry Intelligence: Technical Approach Report

*Covers: model choices, data pipeline, prompt engineering and architecture, evaluation, and execution plan*

## 1. Problem Statement

Cynbit receives free-text project inquiries covering AI, web and mobile, cloud, CRM, analytics and marketing work. Manually reading, categorising, qualifying and replying to each one is slow and inconsistent. The goal is a tool that turns a raw inquiry into: a service category, detected requirements, a transparent priority score, clarification questions and an editable discovery-stage draft, **with a human always in the loop**.

## 2. Current PoC Overview

The PoC is a Streamlit app (`app.py`) using scikit-learn and pandas. Components:

| Component | Implementation | Nature |
|---|---|---|
| Category classifier | TF-IDF (1-2 grams) + Logistic Regression | Trained ML model |
| Requirement tagging | Keyword groups | Rule-based |
| Priority score | Additive points with visible reasons | Heuristic |
| Clarification questions | Rules on missing information | Rule-based |
| Proposal draft | Fixed template filled with extracted fields | Template |
| Exports | TXT draft and CSV analysis record | Utility |

## 3. Architecture

**Current (Phase 1):**

Inquiry text and optional lead context (Streamlit UI) -> text preprocessing inside the TF-IDF vectoriser -> classifier (category and probability) -> rule engines (requirements, score, questions) -> template renderer (draft) -> download as TXT and CSV.

**Proposed (Phase 2 and 3):**

Inquiry intake (web form or email) -> API service (FastAPI) -> classifier plus embedding model -> retrieval over Cynbit's past proposals and service documents (vector store) -> LLM drafting with guardrails -> human review screen -> CRM push (Salesforce or HubSpot) -> feedback log used to retrain.

## 4. Data Pipeline

1. **Source:** currently 36 hand-written synthetic examples (6 per category) embedded in the code, plus `sample_inquiries.csv` for testing. Target: 100-200 or more anonymised real inquiries labelled by the sales team.
2. **Cleaning:** strip signatures and email threads, remove personal data (names, phone numbers, emails), normalise whitespace.
3. **Labelling:** one primary category per inquiry, with optional secondary labels, since real inquiries often span categories (for example an AI chatbot plus CRM integration).
4. **Splitting:** stratified train and test split, or cross-validation given the small size.
5. **Versioning:** store the dataset as CSV with a version and date; keep a changelog of labelling decisions.
6. **Feedback loop:** when a reviewer corrects a category or score, log it as a new labelled example.

## 5. Model Choices and Rationale

| Option | Pros | Cons | Verdict |
|---|---|---|---|
| TF-IDF + Logistic Regression | Fast, interpretable, works on small data, no API cost, runs offline | Weak on synonyms and context | **Chosen for PoC** |
| Sentence embeddings + Logistic Regression | Handles paraphrases better, still lightweight | Needs a pretrained model download | **Recommended next step** |
| Fine-tuned transformer | Strong accuracy with enough data | Needs hundreds of labelled examples, heavier | Later, if data allows |

| Option | Pros | Cons | Verdict |
|---|---|---|---|
| Zero-shot or few-shot LLM classifier | No training data needed, handles multi-label | Cost, latency, privacy, less deterministic | Good as fallback or for labelling help |

**Why Logistic Regression first:** with so little data, a simple linear model is easiest to inspect (top-weighted terms per class) and least likely to be mistaken for a validated system. `class_weight=balanced` guards against future label imbalance.

**Confidence handling:** the predicted probability is **not calibrated**. Plan: add a low-confidence threshold (for example below a tuned cut-off route to General / Needs Discovery) and later apply probability calibration once enough data exists.

## 6. Rule-Based Components

**Priority score (heuristic, not a conversion probability):**

| Signal | Points |
|---|---|
| Baseline | +25 |
| Integration detail mentioned | +15 |
| Business context mentioned | +10 |
| Budget defined | +20 |
| Timeline defined | +15 |
| Contact email provided | +10 |
| Detailed inquiry (35 or more words) | +5 |

Bands: High at 70 or above, Medium at 45 or above, otherwise Low. Every score shows its reasons so a reviewer can disagree with it. Once conversion outcomes exist, this can be replaced or complemented by a trained model and validated against actual win rates.

## 7. Prompt Engineering and LLM Architecture (Phase 2)

The current draft is template-based and does not call an LLM. For a richer draft, the proposed design is **retrieval-augmented generation with strict guardrails**.

**Pipeline:**

1. Embed the inquiry and retrieve the top relevant chunks from Cynbit's service descriptions and approved past proposals.
2. Pass the inquiry, extracted requirements, category and retrieved context to the LLM.
3. Request structured JSON output (objective, suggested approach, assumptions, open questions, risks), validate the schema, then render into the draft template.
4. Show the result to a human reviewer before anything leaves the system.

**Example system prompt (outline):**

> You are an assistant that prepares an internal discovery-stage summary for Cynbit's sales team. Use only the provided inquiry and retrieved company context. Do not state prices, delivery dates or commitments. If information is missing, list it under open questions instead of guessing. Respond only in the given JSON schema.

**Prompt-engineering practices:**

- Few-shot examples of good summaries for each category.
- Explicit refusal rules: no pricing, no guarantees, no invented client names.
- Low temperature for consistency.
- Prompt injection defence: treat inquiry text as untrusted data, never as instructions.
- Log prompts and outputs (with personal data removed) for audit and improvement.

## 8. Evaluation Plan

| Component | Metric | Method |
|---|---|---|
| Classifier | Accuracy, macro-F1, confusion matrix | Held-out real inquiries, cross-validation |
| Confidence | Calibration curve, fraction routed to manual review | Reliability plot |
| Requirement tagging | Precision and recall per tag | Manually labelled sample |
| Priority score | Rank correlation with sales team ratings; later, win rate by band | Reviewer comparison, then outcome tracking |

| Component | Metric | Method |
|---|---|---|
| Drafts | Reviewer rating (accuracy, tone, usefulness), edit distance | Blind review by sales staff |
| Business | Time to first response, share of drafts used with light edits | Before and after comparison |

## 9. Known Limitations and Recommended Fixes in the Current Code

These were found by reading `app.py` and should be fixed before any real use:

1. **Tiny, synthetic training set (36 examples).** Reported confidence is not meaningful as a performance measure. Replace with real labelled data.
2. **Substring keyword matching causes false positives.** For example the keyword `ai` also matches words such as email or maintain, wrongly adding an AI / chatbot tag. Fix with word-boundary regex (the `re` module is already imported but unused).
3. **`CATEGORY_KEYWORDS` is defined but never used.** Either use it as a fallback or remove it.
4. **No General / Needs Discovery routing.** `SERVICE_MAP` has this entry, but the classifier always picks one of six categories. Add a confidence threshold that routes vague inquiries there.
5. **`datetime.utcnow()` is deprecated** in recent Python versions. Use `datetime.now(timezone.utc)`.
6. **Personal data in CSV export.** The exported record contains name, company and email in plain text; document handling rules and consider masking.
7. **Single-label output.** Real inquiries often span categories; show the top two categories with probabilities.
8. **Inquiry text is pasted into the draft verbatim.** Summarise or trim to avoid carrying sensitive details forward.

## 10. Security, Privacy and Governance

- Use only fictional or anonymised data in demos.
- If an external LLM API is used, review the provider's data policy and send only minimised, non-sensitive text.

- Role-based access to the review screen and logs.
- Every draft is marked as not a quotation; an authorised person must approve scope and pricing.
- Keep an audit trail of who approved what.

## 11. Execution Plan

| Week | Milestone | Deliverable |
|---|---|---|
| 1 | Fix known code issues (section 9); add word-boundary matching, low-confidence routing, top-2 categories | Updated PoC |
| 2 | Collect and label real anonymised inquiries; build train and test split | Labelled dataset v1 |
| 3 | Evaluate baseline; try sentence-embedding classifier; compare | Evaluation notebook and metrics table |
| 4 | Reviewer feedback session with sales team; tune score weights | Feedback log |
| 5-6 | Build RAG drafting prototype with schema-validated LLM output | Phase 2 demo |
| 7 | Wrap in FastAPI; add logging and review workflow | API and review UI |
| 8 | CRM integration pilot and final evaluation | Pilot report, go or no-go recommendation |

## 12. Risks and Mitigations

| Risk | Mitigation |
|---|---|
| Not enough real labelled data | Start with active learning: reviewers correct predictions, which become training data |
| Model over-trusted by staff | Show uncertainty clearly; low-confidence routing; training for reviewers |

| Risk | Mitigation |
| --- | --- |
| LLM hallucination in drafts | Retrieval grounding, JSON schema, no-pricing rule, mandatory human review |
| Data leakage to third parties | Data minimisation, provider review, option of self-hosted model |
| Score gaming or bias | Keep score transparent; avoid sensitive attributes; review periodically |

## 13. Tech Stack Summary

- **Language and UI:** Python 3.10-3.11, Streamlit
- **ML:** scikit-learn (TF-IDF, Logistic Regression), optional sentence-transformers for embeddings
- **Data:** pandas, CSV (later a database)
- **Phase 2:** FastAPI, a vector store, an LLM API or self-hosted model
- **Delivery:** GitHub repository, requirements file, README with setup steps